Carleton Smart Course Scheduler

SYSC 4907 Capstone Project

Helping students check prerequisites and build conflict-free schedules

January 2026

The Problem

Course registration at Carleton is a pain

Checking Prerequisites

- Open your audit, open the calendar, go back and forth
- “Wait, do I have SYSC 3303 or not?”
- Takes a while if you’re picking 5-6 courses

Info is Everywhere

- Carleton Central for one thing
- Course calendar for another
- Department sites for elective lists

Building a Schedule

- No way to check conflicts automatically
- Try a section, doesn’t work, try another
- Hope you don’t mess up LEC/LAB/TUT

Actually Registering

- Hunt down CRNs one by one
- Copy them into Central
- Pray nothing conflicts

The Old Way: Checking Prerequisites

Cross-referencing your audit with the course calendar

1. Your Course Audit

SUMMARY OF CREDITS AND CGPA
Engineering Program Only
EARNED: 17.00 Credits
16.50 CGPA Credits 8.78 CGPA

NO COMP. STUDIES...

What have I completed?

2. Course Calendar

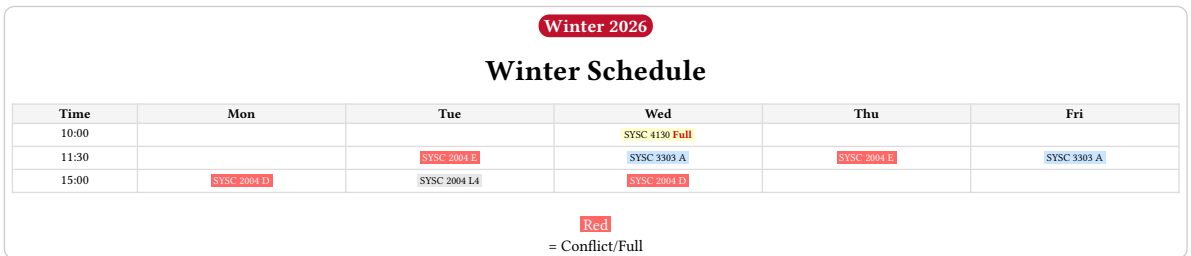
SYSC 2004 [0.5 credit]
Object-Oriented Software Development
Prereq: SYSC 1006 or SYSC 2006, and 2nd-year standing. **SYSC 2006 [0.5 credit]**
Foundations of Imperative Programming

What do I need?

Students must manually cross-reference these for every course

The Old Way: Building a Schedule

Conflicts only appear after you try to register



The Problem

- Pick courses and sections
- Build what looks good, try to register… **Conflict!**
- Or worse: section is **full**

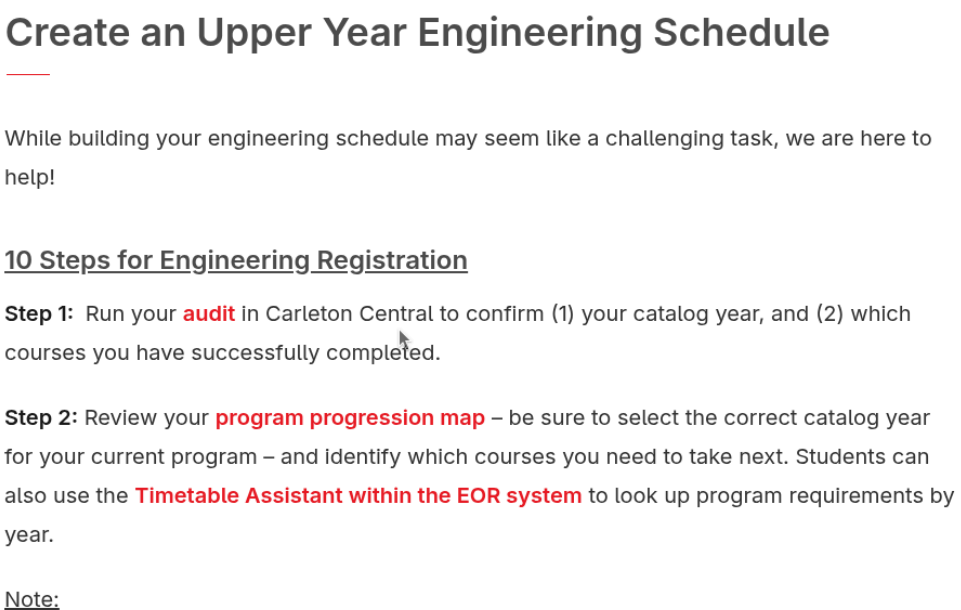
Result: Start over, hope next combo works

No Way to Know Until You Try

No automatic conflict detection

The Old Way: Info is Everywhere

10 steps just to register for courses?



Create an Upper Year Engineering Schedule

While building your engineering schedule may seem like a challenging task, we are here to help!

10 Steps for Engineering Registration

Step 1: Run your **audit** in Carleton Central to confirm (1) your catalog year, and (2) which courses you have successfully completed.

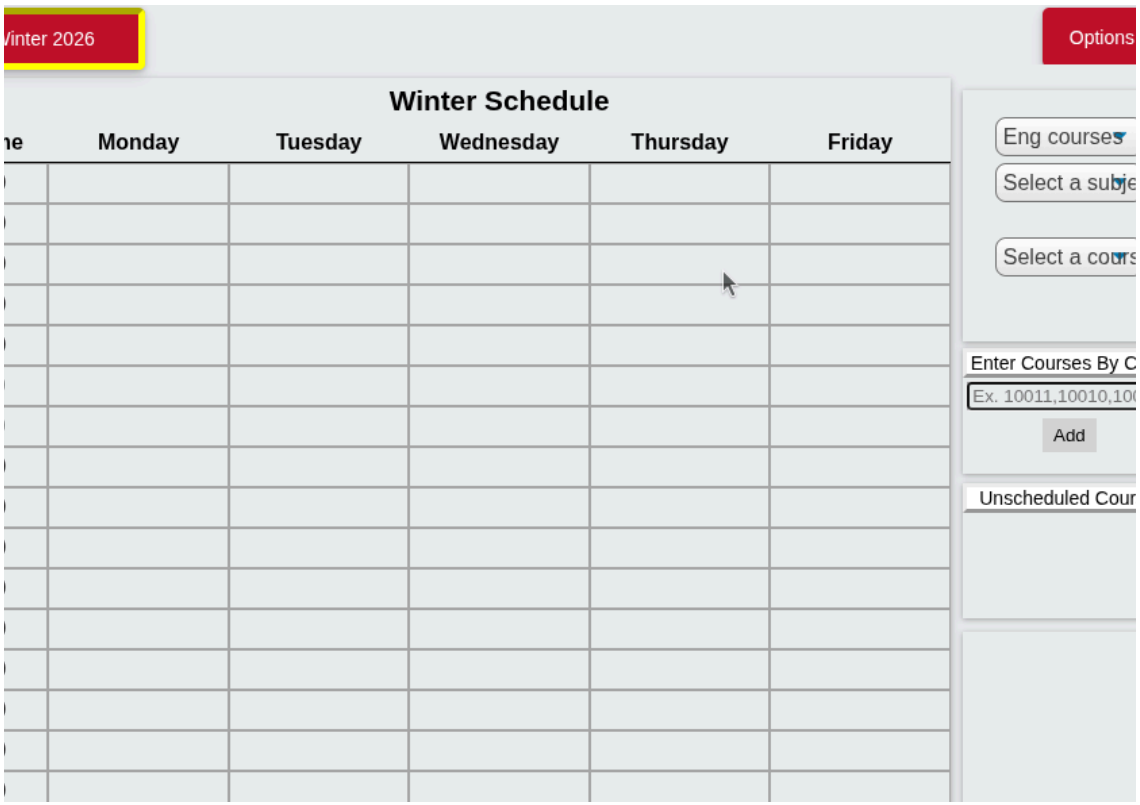
Step 2: Review your **program progression map** – be sure to select the correct catalog year for your current program – and identify which courses you need to take next. Students can also use the **Timetable Assistant within the EOR system** to look up program requirements by year.

Note:

Students must navigate multiple disconnected systems just to build a schedule

The Old Way: No Search

You can't just type a course name



The Problem

- No search bar at all
- Must click through dropdown menus
- "Select a subject" → "Select a course"
- Or enter CRNs manually if you know them

What This Means

- Can't type "SYSC 4001" and find it
- Have to browse through long lists
- Slows down the whole process

Even basic search functionality is missing from Carleton's scheduler

Our Solution: Engineering Overview

Full-stack system consolidating fragmented data and automating manual processes

System Architecture

Frontend — React + TypeScript
Audit upload, course search, schedule viewer

↓ REST API ↓

Backend — Express + Node.js
Validation engine, schedule generator

↓ Prisma ORM ↓

Database — PostgreSQL
Courses, prerequisites, sections, students

Core Engineering Problems

1. **Academic Audit Parsing** — Extract data from HTML
2. **Prerequisite Representation** — AND/OR logic as trees
3. **Schedule Generation** — Constraint satisfaction
4. **Data Acquisition** — Scraping, electives, terms

Next: Implementation walkthrough — what we built and why

Section 1: Academic Audit Parsing

The foundation — everything else depends on knowing what you've completed

Why This Comes First

Before we can do anything useful, we need to know:

- Which courses has the student completed?
- What grades did they get?
- What program are they in?
- What requirements remain unfulfilled?

Every other feature depends on this data:

- Prerequisite validation needs completed courses
- Elective suggestions need program info
- Schedule generation needs to know what's left

The Challenge

No API exists.
Carleton Central provides audits as HTML and PDF only.
We must extract structured data from unstructured markup.

What We Extract

- Student ID (9 digits)
- Student name
- Program (e.g., "Software Engineering")
- Completed courses with grades
- In-progress courses (current term)
- Requirement status (fulfilled/unfulfilled)

Audit Parser: Data Structures

TypeScript interfaces that model the extracted data

StudentAuditData Interface

```
interface StudentAuditData {
  studentId: string; name: string; program: string
  completedCourses: Array<{ courseId: string; grade: string }>
  currentCourses: Array<{ courseId: string; term: string }>
  requirements: RequirementInfo[]
}
```

RequirementInfo Interface

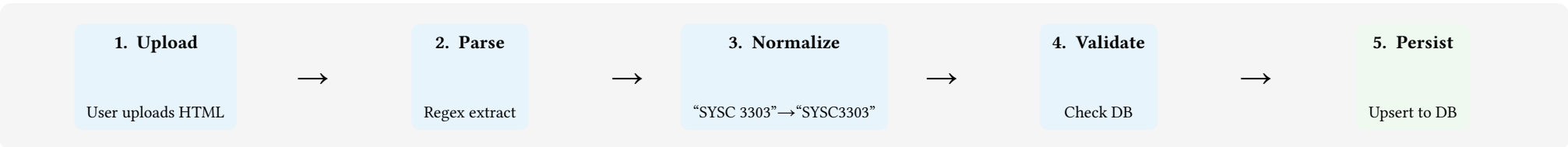
```
interface RequirementInfo {
  title: string; status: 'OK' | 'NO' | 'IP' | 'NONE'
  creditsRequired: number; creditsCompleted: number
  creditsInProgress: number; courseOptions: string[]
}
```

Why These Fields?

- **status**: track completion state
- **creditsRequired vs Completed**: calculate progress
- **courseOptions**: which courses satisfy requirement

Audit Parser: Pipeline Architecture

From raw HTML upload to structured database records



Key Design: Upsert

- Use `prisma.student.upsert()` for re-uploads
- Existing → update; New → create
- Idempotent: safe to upload multiple times

Database Operations

1. Upsert Student record
2. Delete old CourseCompletions
3. Insert new CourseCompletions
4. Link to existing Course records

Audit Parser: Regex Patterns Targeted patterns for each data field we need to extract

Core Extraction Patterns

Field	Regex Pattern	Example Match
Student ID	`/(\d{9})/`	“123456789”
Name	`/([A-Z][a-z]+), ([A-Z][a-z]+)/`	“Smith, John” → “John Smith”
Program	`/Bachelor of Engineering - ([^<]+)/`	“Software Engineering”
Course Row	`/<tr[^>]*class="takenCourse"[^>]*>/g`	Matches each course row

Course Extraction (Complex)

```
/<tr[^>]*class="takenCourse"[^>]*>
  [\s\S]*?<td[^>]*class="term"[^>]*>
    ([^<]*)<\/td>
  [\s\S]*?<td[^>]*class="course"[^>]*>
    ([A-Z]{3,4}\s+\d{4})<\/td>
  [\s\S]*?<td[^>]*class="grade"[^>]*>
    ([^<]+)<\/td>/g
```

Why This Pattern?

- `[\s\S]*?` — non-greedy match across newlines
- Captures: term, course code, grade
- Class selectors target stable HTML structure
- Global flag `/g` finds all course rows

Audit Parser: Requirement Extraction

Parsing subrequirements to identify what's still needed

HTML Structure We Parse

```
<div class="subrequirement" rqdHours="0.50">
  <span class="status Status_NO">
  <span class="subreqTitle">
    COMPLEMENTARY STUDIES, CCDP 2100
  </span>
  ...
  <span class="hours">0.00</span> cr. Complete
</div>
```

Extract: required credits, completed credits, status, and course options from each block.

Status Detection Logic

```
let status: 'OK'|'NO'|'IP'|'NONE' = 'NO'
if (statusClass.includes('OK')) status = 'OK'
else if (statusClass.includes('IP')) status = 'IP'
```

Elective Category Detection

- "complementary studies" ⟶ `COMPLEMENTARY_STUDIES`
- "computer science elective" ⟶ `CS_ELECTIVE`
- "basic science" ⟶ `SCIENCE_ELECTIVE`
- These map to lists of valid courses later

Audit Parser: PDF Support

Supporting both HTML exports and PDF text extraction

PDF Line Format

```
21FA   CHEM 1101   0.50   A+
21WN   MATH 1004   0.50   B
25WN   SYSC 4907   0.50   CUR
```

PDF Pattern

```
/^\s*(\d{2}(?:FA|WN|SU))\s+
  ([A-Z]{3,4})\s+(\d{4})\s+
  ([\d.]+)\s+([A-Z+\-]+)/gm
```

Grade Filtering Logic

Not all rows are valid completions:

```
if (grade === 'CUR') {
  currentCourses.push({ courseId, term })
} else if (['F','X','UNS','NR'].includes(grade)) {
  continue  // Failed or withdrawn
} else {
  completedCourses.push({ courseId, grade })
}
```

Key: We use `Set` to dedupe — same course may appear multiple times if retaken.

Audit Parser: Database Persistence

Saving parsed data with Prisma ORM

Student Upsert

```
const student = await prisma.student.upsert({
  where: { studentId: auditData.studentId },
  update: { name: auditData.name, program: auditData.program },
  create: { studentId: auditData.studentId,
            name: auditData.name, program: auditData.program }
})
```

Upsert = Update if exists, Insert if new. Handles re-uploads gracefully.

Course Completion Storage

```
await prisma.courseCompletion.deleteMany({ where: { studentId: student.id } })
for (const completion of completedCourses) {
  const course = await prisma.course.findUnique({ where: { id: completion.courseId } })
  if (course) {
    await prisma.courseCompletion.create({
      data: { studentId: student.id, courseId: completion.courseId, grade: completion.grade }
    })
  }
}
```

Audit Parser: Trade-offs & Limitations

Design decisions and their consequences

Approach Comparison

Approach	Pros	Cons
Regex	Fast, simple, no deps	Brittle to HTML changes
DOM (cheerio)	Robust, semantic	Heavier, slower
Official API	Reliable	Doesn't exist

Why We Chose Regex

- O(n) single-pass parsing
- No external dependencies
- Patterns target stable class names
- Easy to update if structure changes

Known Limitations

1. **HTML structure dependency** — if Carleton redesigns their audit page, patterns break
2. **No real-time sync** — student must re-upload after completing courses
3. **Course existence check** — courses not in our DB are skipped (logged as warnings)

Mitigation Strategy

- Fallback patterns for common variations
- Comprehensive logging for debugging
- Could migrate to DOM parser if needed

Section 2: Prerequisite Representation

Modeling complex AND/OR logic for automated validation

The Core Problem

Prerequisites aren't simple lists. They have **logic**:

> **SYSC 4001:**
> "SYSC 3303 **and** (SYSC 3310 **or** SYSC 3320)"
>
> **SYSC 4805:**
> "Fourth-year standing **or** permission"

We need to:

- Store this logic in a database
- Evaluate it against a student's transcript
- Show clear feedback (pass/fail + what's missing)

Requirements

1. **Persist in relational DB** — can't just store strings
2. **Evaluate programmatically** — no human interpretation
3. **Support multiple node types:**
   - Course requirements (SYSC3303)
   - Year standing (Fourth-year)
   - Special permission
   - Grade requirements (C- minimum)
4. **Handle nested logic** — AND inside OR inside AND
5. **Provide actionable feedback** — list missing courses

Prerequisite Schema: Database Design

Self-referential tree structure in PostgreSQL via Prisma

PrerequisiteNode Model

```
model PrerequisiteNode {
  id               String   @id @default(cuid())
  courseId         String   // Which course this prereq belongs to
  nodeType         String   // AND | OR | COURSE | YEAR_STANDING | PERMISSION
  requiredCourseId String?  // FK to Course (for COURSE nodes)
  requiredGrade    String?  // e.g., "C-"
  requiredYear     String?  // FIRST_YEAR | SECOND_YEAR | etc.
  // Tree structure (self-reference)
  parentId         String?
  parent           PrerequisiteNode?  @relation("NodeHierarchy")
  children         PrerequisiteNode[] @relation("NodeHierarchy")
  course           Course   @relation("CoursePrerequisites")
}
```

Why Self-Reference?

- Trees need parent-child links
- Root nodes have `parentId = null`
- Prisma handles FK automatically

Node Types

Type	Meaning
AND	All children must pass
OR	Any child must pass
COURSE	Specific course required
YEAR	Minimum year level

Prerequisite Parser: Expression → Tree Converting human-readable expressions to node trees

Input Format (from course calendar)

```
"SYSC3303 && (SYSC3310 || SYSC3320)"
"FOURTH_YEAR || PERMISSION"
"COMP1006[C-] && MATH1004"
```

Parser Strategy

1. Split on && first (lower precedence)
2. Split on || within groups
3. Handle parentheses for nesting
4. Parse individual terms

Parsing Logic

```
function parsePrerequisiteExpression(expr: string): ParsedNode {
  // Handle simple AND (no OR present)
  if (expr.includes(' && ') && !expr.includes(' || ')) {
    const parts = expr.split(' && ')
    return { type: 'AND', children: parts.map(parseSimpleTerm) }
  }
  // Handle simple OR (no AND present)
  if (expr.includes(' || ') && !expr.includes(' && ')) {
    const parts = expr.split(' || ')
    return { type: 'OR', children: parts.map(parseSimpleTerm) }
  }
  // Handle parentheses - recursive
  if (expr.includes('(')) return parseComplexExpression(expr)
  return parseSimpleTerm(expr)
}
```

Prerequisite Parser: Term Recognition

Identifying different types of requirements

parseSimpleTerm Function

```
function parseSimpleTerm(term: string): ParsedNode {
  term = term.trim().replace(/[()]/g, '')
  if (['FIRST_YEAR', 'SECOND_YEAR',
       'THIRD_YEAR', 'FOURTH_YEAR'].includes(term))
    return { type: 'YEAR_STANDING', value: term }
  if (term === 'PERMISSION') return { type: 'PERMISSION' }
  const gradeMatch = term.match(/^([A-Z]{3,4}\d{4})\[([A-Z\-+]*)\]$/)
  if (gradeMatch)
    return { type: 'COURSE', value: gradeMatch[1], grade: gradeMatch[2] }
  if (term.match(/^[A-Z]{3,4}\d{4}$/))
    return { type: 'COURSE', value: term }
  return { type: 'PERMISSION' }  // Unknown - fallback
}
```

Example Transformations

Input	Output Node
"SYSC3303"	{type: 'COURSE', value: 'SYSC3303'}
"COMP1006[C-]"	{type: 'COURSE', value: 'COMP1006', grade: 'C-'}
"FOURTH_YEAR"	{type: 'YEAR_STANDING', value: 'FOURTH_YEAR'}
"PERMISSION"	{type: 'PERMISSION'}

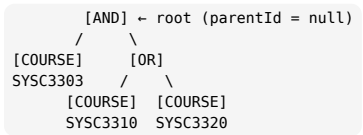
Fallback: Unknown terms become PERMISSION nodes to avoid FK errors.

Prerequisite Storage: Building the Tree Recursively creating nodes in PostgreSQL

Recursive Tree Creation

```typescript
async function createPrerequisiteTree(
  courseId: string, parsedNode: ParsedNode, parentId?: string
): Promise<string> {
  const node = await prisma.prerequisiteNode.create({
    data: {
      courseId, nodeType: parsedNode.type,
      requiredCourseId: parsedNode.type === 'COURSE' ? parsedNode.value : null,
      requiredYear: parsedNode.type === 'YEAR_STANDING' ? parsedNode.value : null,
      requiredGrade: parsedNode.grade || null,
      parentId: parentId || null
    }
  })
  if (parsedNode.children) {
    for (const child of parsedNode.children)
      await createPrerequisiteTree(courseId, child, node.id)
  }
  return node.id
}
```

Visual Example Expression: `"SYSC3303 && (SYSC3310 || SYSC3320)"`



Each box = one row in `prerequisite_nodes`. Links via `parentId` FK

Prerequisite Validation: Evaluation Algorithm Recursive tree traversal with boolean logic

Core Evaluation Function

```
async function evaluatePrerequisiteNode(
  node: PrerequisiteNodeData, transcript: StudentTranscript,
  allNodes: PrerequisiteNodeData[]
): Promise<EvaluationResult> {
  switch (node.nodeType) {
    case 'AND': {
      const children = allNodes.filter(n => n.parentId === node.id)
      const results = await Promise.all(
        children.map(c => evaluatePrerequisiteNode(c, transcript, allNodes))
      )
      const failed = results.filter(r => !r.isValid)
      return { isValid: failed.length === 0,
               missingItems: failed.flatMap(r => r.missingItems) }
    }
    case 'OR': { /* Returns true if ANY child passes */ }
    case 'COURSE': { /* Check if course in transcript */ }
  }
}
```

Node Type Handlers

Type	Logic
AND	All children must return `isValid: true`
OR	At least one child must return `isValid: true`
COURSE	Check `transcript.completedCourses.include`
YEAR	Compare `transcript.currentYear >= requiredYear`
PERM	Check `transcript.hasSpecialPermission`

Complexity **Time:** O(n) where n = nodes; **Space:** O(h) for recursion. Typical tree: 5-15 nodes

Prerequisite Validation: Grade Requirements Some courses require a minimum grade, not just completion

Grade Scale Implementation

```
function meetsGradeRequirement(
  actualGrade: string | undefined, requiredGrade: string
): boolean {
  if (!actualGrade) return false
  const gradeValues: Record<string, number> = {
    'A+': 12, 'A': 11, 'A-': 10, 'B+': 9, 'B': 8, 'B-': 7,
    'C+': 6, 'C': 5, 'C-': 4, 'D+': 3, 'D': 2, 'D-': 1, 'F': 0
  }
  const actualVal = gradeValues[actualGrade]
  const requiredVal = gradeValues[requiredGrade]
  if (actualVal === undefined || requiredVal === undefined) return true
  return actualVal >= requiredVal
}
```

Usage in COURSE Node Evaluation

```
case 'COURSE': {
  const hasCourse = transcript.completedCourses
    .includes(node.requiredCourseId)
  if (!hasCourse)
    return { isValid: false, missingItems: [node.requiredCourseId] }
  if (node.requiredGrade) {
    const actual = transcript.courseGrades?.[node.requiredCourseId]
    if (!meetsGradeRequirement(actual, node.requiredGrade))
      return { isValid: false, gradeIssues:
        [`${node.requiredCourseId}: need ${node.requiredGrade}, have ${actual}`] }
  }
  return { isValid: true }
}
```

Prerequisite Validation: User Feedback

Providing actionable information, not just pass/fail

ValidationResult Interface

```
interface PrerequisiteValidationResult {
  isValid: boolean
  reason?: string
  missingCourses?: string[]    // ["SYSC3303", "SYSC3310"]
  yearRequirement?: string     // "FOURTH_YEAR"
  needsPermission?: boolean
  gradeIssues?: string[]       // ["COMP1006: need C-, have D+"]
  expressionEvaluated: string  // Original expression
}
```

We return **all** missing items, not just the first — helps students plan.

Example Output

Course: SYSC 4001
Status: FAIL
Required: `SYSC3303 && (SYSC3310 || SYSC3320)`
Missing: SYSC3303

Course: SYSC 4907
Status: PASS
Required: `FOURTH_YEAR`

Section 3: Schedule Generation

Finding conflict-free timetables using backtracking

The Combinatorial Problem

A typical term schedule involves:

- 5 courses
- 3-4 sections per course
- Each section may have LEC + LAB + TUT

Worst case: 4 sections × 5 courses = 4^5 = **1,024 combinations**
With LEC/LAB/TUT: easily **10,000+ combinations**

Manual checking is tedious and error-prone. We need an algorithm.

Requirements

1. **Find ALL valid schedules** — not just one
2. **No time conflicts** — classes can't overlap
3. **Include all components** — LEC + LAB + TUT
4. **Fast enough** — user shouldn't wait > 1-2 seconds
5. **Bounded output** — don't return 10,000 schedules

Algorithm Choice: Backtracking

- Systematic search of solution space
- **Prune early** when conflict detected
- Guaranteed to find all solutions
- Much faster than brute force in practice

Schedule Generation: Data Structures

TypeScript interfaces for sections, schedules, and time slots

Section Interface

```typescript
interface Section {
  id: string
  courseId: string      // "SYSC3303"
  crn: string | null    // "12345"
  sectionCode: string   // "A"
  sectionType: string   // "LEC" | "LAB" | "TUT"
  days: string | null   // "MWF" or "TR"
  startTime: string | null  // "0835"
  endTime: string | null    // "0925"
  capacity: number | null
}
```

Schedule & TimeSlot

```typescript
interface TimeSlot {
  day: string; startTime: string; endTime: string
}
interface ScheduleSelection {
  courseId: string; section: Section
}
interface GeneratedSchedule {
  id: number
  selections: ScheduleSelection[]
  hasConflicts: boolean
  prerequisiteInfo?: {
    [courseId: string]: { isValid: boolean; missingCourses?: string[] }
  }
}
```

Schedule Generation: Time Handling

Converting database formats to comparable time slots

Day Abbreviation Expansion

```
function parseDays(daysStr: string | null): string[] {
  if (!daysStr) return []
  const dayMap: Record<string, string> = {
    'M': 'Monday', 'T': 'Tuesday', 'W': 'Wednesday',
    'R': 'Thursday', 'F': 'Friday'  // R for thuRsday
  }
  return daysStr.split('').map(d => dayMap[d] || d).filter(d => d)
}
// "MWF" → ["Monday", "Wednesday", "Friday"]
// "TR"  → ["Tuesday", "Thursday"]
```

Section → TimeSlots

```
function getTimeSlots(section: Section): TimeSlot[] {
  if (!section.days || !section.startTime || !section.endTime) {
    return []  // Online/async sections
  }
  const days = parseDays(section.days)
  return days.map(day => ({
    day, startTime: section.startTime!, endTime: section.endTime!
  }))
}
// Section with days="MWF", start="0835", end="0925"
// → [{ day: "Monday", startTime: "0835", endTime: "0925" },
//    { day: "Wednesday", ... }, { day: "Friday", ... }]
```

Schedule Generation: Conflict Detection

The O(1) interval overlap check at the heart of the algorithm

Time Conflict Logic

```typescript
function hasTimeConflict(slot1: TimeSlot, slot2: TimeSlot): boolean {
  if (slot1.day !== slot2.day) return false  // Different days
  // Parse "0835" → 515 minutes since midnight
  const parseTime = (time: string): number => {
    const hours = parseInt(time.substring(0, 2))
    const minutes = parseInt(time.substring(2, 4))
    return hours * 60 + minutes
  }
  const start1 = parseTime(slot1.startTime), end1 = parseTime(slot1.endTime)
  const start2 = parseTime(slot2.startTime), end2 = parseTime(slot2.endTime)
  return start1 < end2 && start2 < end1  // Interval overlap
}
```

Why This Formula Works

```
Overlap if: start1 < end2 AND start2 < end1

Case 1: [===1===]
              [===2===]  ✓ Overlap
Case 2: [===1===]
                  [===2===]  ✗ No overlap
```

Time: O(1) per comparison
Called: Many times during backtracking

Schedule Generation: Section Grouping

Handling courses with multiple component types (LEC + LAB + TUT)

Group by Section Type

```
function groupSectionsByType(sections: Section[]): Record<string, Section[]> {
  const grouped: Record<string, Section[]> = {}
  for (const section of sections) {
    if (!grouped[section.sectionType]) grouped[section.sectionType] = []
    grouped[section.sectionType].push(section)
  }
  return grouped
}
// Input: [LEC-A, LEC-B, LAB-L1, LAB-L2, TUT-T1]
// Output: { "LEC": [LEC-A, LEC-B], "LAB": [LAB-L1, LAB-L2], "TUT": [TUT-T1] }
```

Generate All Combinations

```
function generateSectionCombinations(courseId: string,
  sectionsByType: Record<string, Section[]>): Section[][] {
  const types = Object.keys(sectionsByType)
  if (types.length === 1) return sectionsByType[types[0]].map(s => [s])
  const combinations: Section[][] = []
  function build(typeIdx: number, current: Section[]) {
    if (typeIdx === types.length) { combinations.push([...current]); return }
    for (const section of sectionsByType[types[typeIdx]]) {
      current.push(section); build(typeIdx + 1, current); current.pop()
    }
  }
  build(0, [])
  return combinations
}
```

2 LECs × 2 LABs × 1 TUT = 4 combinations per course

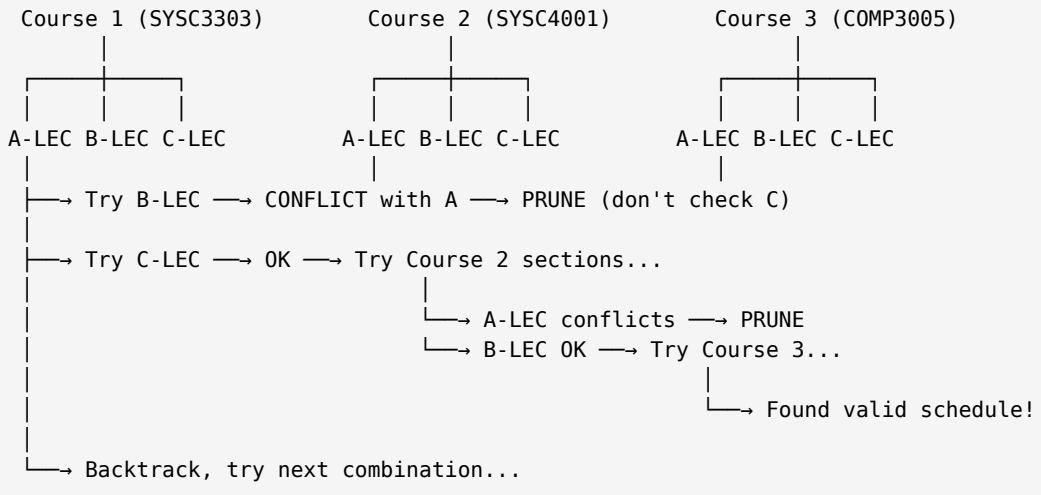
Schedule Generation: Backtracking Algorithm

The recursive search with early pruning

```
function generateSchedulesRecursive(courseSections: { courseId: string; sections: Section[] }[],
  currentIndex: number, currentSelections: ScheduleSelection[], maxSchedules: number, result: GeneratedSchedule[]
): void {
  if (currentIndex === courseSections.length) {  // Base case: all courses scheduled
    result.push({ id: result.length + 1, selections: [...currentSelections], hasConflicts: false }); return
  }
  if (result.length >= maxSchedules) return  // Early termination
  const { courseId, sections } = courseSections[currentIndex]
  const combinations = generateSectionCombinations(courseId, groupSectionsByType(sections))
  for (const sectionCombination of combinations) {
    if (hasInternalConflicts(sectionCombination)) continue
    let hasConflict = false
    for (const section of sectionCombination) {
      if (hasConflictWithSelections(section, currentSelections)) { hasConflict = true; break } // PRUNE
    }
    if (!hasConflict) {
      for (const section of sectionCombination) currentSelections.push({ courseId, section })
      generateSchedulesRecursive(courseSections, currentIndex + 1, currentSelections, maxSchedules, result)
      for (let i = 0; i < sectionCombination.length; i++) currentSelections.pop() // Backtrack
    }
  }
}
```

Schedule Generation: Algorithm Visualization

How backtracking explores and prunes the search tree



Why Pruning Helps

- Conflict at Course 1 ⟶ skip all Course 2/3 combinations
- Exponential reduction in practice
- Most branches get pruned early

Complexity

- **Worst case:** O(b^d) where b=sections, d=courses
- **Average case:** Much better due to pruning
- **Real performance:** < 1 second for 5 courses

Schedule Generation: API Integration

REST endpoint with prerequisite validation

POST /schedules/generate

```
router.post('/generate', async (req, res) => {
  const { courseIds, term, maxSchedules = 50 } = req.body
  if (courseIds.length > 8) return res.status(400).json({error:'Max 8 courses'})
  const courseSections = await Promise.all(
    courseIds.map(async (id) => ({
      courseId: id,
      sections: await prisma.section.findMany({ where: { courseId: id, term } })
    }))
  )
  const schedules: GeneratedSchedule[] = []
  generateSchedulesRecursive(courseSections, 0, [], maxSchedules, schedules)
  res.json({ schedules, count: schedules.length })
})
```

Response Format

```
{ "schedules": [{
    "id": 1,
    "selections": [{ "courseId": "SYSC3303",
      "section": {"crn":"12345","sectionCode":"A","days":"MWF","startTime":"0835"} }],
    "hasConflicts": false
  }],
  "count": 15 }
```

Frontend displays these as timetable grids with CRNs for registration.

Section 4: Data Acquisition The first question: What courses even exist?

Where I Started

Before I could build anything, I needed to answer a basic question:

> *"What courses exist at Carleton, and when are they offered?"*

I wasn't thinking about prerequisites yet. I just needed raw course data:

- What courses exist?
- What sections does each course have?
- What times do they meet?
- Which semester is each course in?

The Problem

Carleton has no public API.

There's no endpoint I can call to get "all courses". The data exists, but it's locked inside their internal systems.

My Thought Process Where does this data already exist in a usable form?

→ **The Schedule Builder** on Carleton Central

→ It somehow loads all sections for a term

→ There must be a backend call I can intercept

Data Acquisition: Timetable Source

Engineering decision: Where to get section/scheduling data

Problem Need all course sections with times, days, CRNs for schedule generation. No public API exists.

Options Considered

Option	Trade-off
Scrape Central pages	Slow, 1000s of requests
PDF timetable parsing	Fragile, format changes
Intercept existing tool	Fast, single request

Chosen Approach Reverse-engineer Schedule Builder via DevTools:



Why It Works Single endpoint returns **all** sections for a term. Same data source Carleton uses.

Limitations Undocumented — could change. Mitigation: modular parser.

Data Acquisition: The First Fetch One curl command, 3,800 sections

The Command

```
curl "https://central.carleton.ca/prod/
      scheduler_server.php?term=202610" > sectiondata.txt
```

What This Gave Me A single file with **3,800 rows**: every dept, course, section type (LEC/LAB/TUT), times, days, CRNs, capacity.

One request = entire term's schedule data

Why This Was Perfect

Structured — Tab-separated, not HTML
Complete — All sections in one file
Easy to parse — Just split on tabs

What I Didn't Get (Yet)

- Prerequisites, course titles
- Elective categories

Separate fetches needed

Data Acquisition: Parsing Section Data

Turning raw TSV into structured database records

Raw Data Format

```
term  CRN    dept  num   sec  type  day  start  end
25WN  12345  SYSC  3303  A    LEC   MWF  0835   0925
25WN  12346  SYSC  3303  L1   LAB   T    1435   1725
```

What Each Row Tells Me

- **SYSC3303** section **A** meets **MWF at 8:35am**
- Lab options: **L1** (Tuesday), **L2** (Thursday)
- CRN **12345** is what students need to register

The Parser I Built

```
function parseTimetableLine(line: string) {
  const parts = line.split('\t')
  return {
    crn: parts[1],
    courseId: `${parts[2]}${parts[3]}`,
    sectionCode: parts[4],
    sectionType: parts[5],
    days: parts[6],
    startTime: parts[7],
    endTime: parts[8]
  }
}
```

Simple split on tabs, map to fields, insert into DB.

Data Acquisition: What the Data Revealed

Understanding Carleton's course structure from the raw data

Course Structure Patterns By looking at the data, I discovered how Carleton structures courses:

SYSC 3303 has:
→ Sections A, B, C (lectures)
→ Each lecture has labs: L1, L2, L3…
COMP 2401 has:
→ Section A (lecture)
→ Labs L1, L2, L3, L4; Tutorials T1, T2
Some courses (like seminars):
→ Just one section, no lab/tut

Why This Matters for Scheduling To register for SYSC 3303, a student needs to pick one lecture section (A, B, or C) and one lab that fits.

Insight: The schedule generator needs to try combinations of LEC + LAB + TUT and check if they fit without time conflicts.

Data Quality

- Some rows had missing times (online/async)
- Some had "TBA" for room/instructor
- Handled these as "no time conflict" cases

Data Acquisition: First Fetch Summary

What the timetable fetch accomplished



What I Had

- ✓ Every course this term
- ✓ All sections, times, CRNs
- ✓ Section types & capacity

Still Needed

- × Course titles
- × Prerequisites
- × Elective mappings

Next: Get prerequisites from course calendar

Data Acquisition: The Next Problem I have courses, but can students actually take them?

The New Question

The timetable data told me **what** courses exist and **when** they meet. But it didn't tell me:

> *"What do I need to have completed before I can take this course?"*

For a scheduler to be useful, it needs to warn students:
"You can't take SYSC 4001 — you're missing SYSC 3303."

Where Prerequisites Live

Carleton publishes all course information at:

```
https://calendar.carleton.ca/undergrad/courses/
```

Each department has a page with:

- Course title and description
- Credit value
- **Prerequisite(s)** — what we need
- Preclusions (courses that conflict)

Sounds simple enough... right?

Data Acquisition: Identifying Courses Finding courses in the HTML is the easy part

The HTML Structure Each course page has a predictable URL pattern:

```
/undergrad/courses/{DEPT}/{COURSE}/
Example: /undergrad/courses/SYSC/SYSC3303/
```

Prerequisite text is in a consistent div:

```
<div class="coursedescadditional">
  Prerequisite(s): COMP 1005 or COMP 1405.
</div>
```

The Fetch Process For each course, I ran:

```
curl "https://calendar.carleton.ca/undergrad/
      courses/COMP/COMP1501/" > comp1501.html
```

Saved 348 course pages locally:

```
courses/sysc/sysc1006.html, sysc3303.html...
```

This part was straightforward. The hard part was next.

Data Acquisition: The Courseblock HTML

Each course on the calendar has a consistent HTML structure

What the HTML Looks Like

```
<div class="courseblock">
  <strong>
    <span class="courseblocktitle">
      <span class="courseblockcode">
        COMP 1405
      </span>
      [0.5 credit]<br>
      Introduction to CS I
    </span>
  </strong><br>
  Introduction to computer science...
</div>
```

Consistent across all 500 courses.

What We Extract From It

HTML Element	Data We Get
`courseblockcode`	Course code (COMP1405)
`[0.5 credit]`	Credit value (0.5)
After ` `	Course title
After `</strong>`	Course description

Key insight: The structure is predictable, so regex can reliably extract each field.

Data Acquisition: Parsing Functions

The regex patterns that extract course data from HTML

Step 1: Find All Course Blocks

```
// Match each <div class="courseblock">
const courseBlockRegex =
  /<div class="courseblock">([\s\S]*?)<\/div>/g
let match: RegExpExecArray | null
while ((match = courseBlockRegex.exec(html))) {
  const blockContent = match[1]
  // Parse this block...
}
```

Pattern: `[\s\S]*?` matches any char (incl. newlines) non-greedily.

Step 2: Extract Course Details

```
const headerRegex =
  /<span class="courseblockcode">
    ([A-Z]{2,5}) (\d{4})
  <\/span>\s+
  \[([0-9.]+)\s+credit[s]?\]<br\/>\s*
  ([^<]+)<\/span>/
const [, dept, num, credits, title] =
  headerRegex.exec(blockContent)
```

Note: ` ` is HTML entity for non-breaking space.

Data Acquisition: Extracting Descriptions

Getting the course description requires a separate pattern

The Description Pattern

```
const descriptionRegex =
  /<\/span><\/strong><br\/>\s*([^<]+)/
const descMatch =
  descriptionRegex.exec(blockContent)
let description = descMatch
  ? descMatch[1].trim()
  : title.trim()
```

The `[^<]+` pattern captures text until the next HTML tag.

Putting It Together

```
courses[courseId] = {
  title: title.trim(),
  credits: parseFloat(creditsStr),
  description: description
}
```

Processing: 30 depts × 15 courses = 500 parsed in batches with delays

Data Acquisition: The JSON Output

All course data merged into a single JSON file

course_prerequisites_simple.json

```
{
  "SYSC1006": {
    "required": "ECOR1031[C-] || ...",
    "restricted": ["COMP2401"],
    "title": "Foundations of Imperative...",
    "credits": 0.5
  },
  "SYSC3303": {
    "required": "SYSC2004 && SYSC2100",
    "restricted": [],
    "title": "Real-Time Concurrent Systems",
    "credits": 0.5
  }
}
```

What Each Entry Contains

Field	Source
`required`	Manual conversion from prose
`restricted`	Preclusions from calendar
`title`	Parsed from courseblock
`credits`	Parsed from courseblock

Single source of truth — seed script reads this to populate the DB.

Total: **348 courses** with complete data

Data Acquisition: JSON to Database

The seed script parses JSON and populates PostgreSQL

seed.ts — Creating Course Records

```
const data = JSON.parse(
  await fs.readFile('course_prerequisites.json')
)
for (const courseId of allCourses) {
  const entry = data[courseId]
  await prisma.course.upsert({
    where: { id: courseId },
    update: { title: entry?.title, ... },
    create: { id: courseId, ... }
  })
}
```

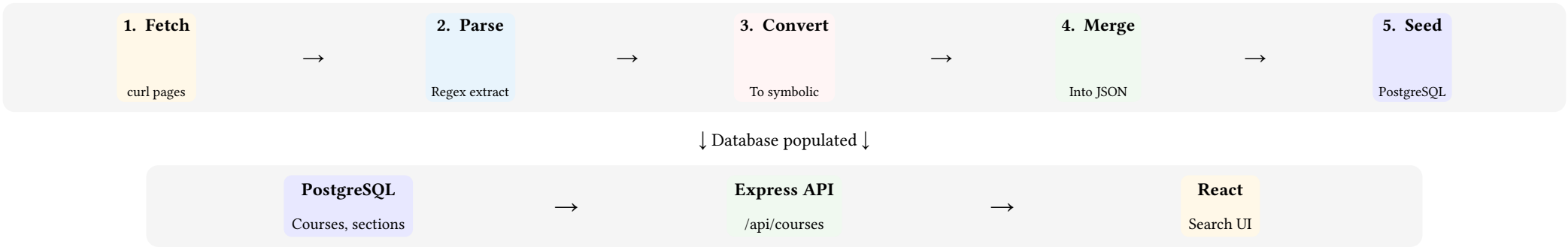
What Gets Created

1. **Course record** — id, title, credits
2. **Prerequisite expression** — symbolic string
3. **Prerequisite tree nodes** — AND/OR/COURSE

Why Upsert?

- Update if exists, create if new
- Safe to run multiple times (idempotent)
- Re-import without duplicates

Data Acquisition: The Complete Pipeline From Carleton's website to the user's screen



Result: User types "SYSC" → API queries DB → Returns courses with titles, credits, section types

Data Acquisition: The Search API

How the frontend gets course data from the database

GET /api/courses/search?q=sysc

```
const courses = await prisma.course.findMany({
  where: {
    OR: [{ id: { contains: q.toUpperCase() } },
         { title: { contains: q } }],
    sections: { some: {} }
  },
  take: 20
})
return { code: course.id, name: course.title,
         credits: course.credits,
         sectionTypes: ["LAB", "LEC"] }
```

API Response


```
{ "results": [
    { "code": "SYSC1006",
      "name": "Foundations of...",
      "credits": 0.5,
      "sectionTypes": ["LAB", "LEC"] },
    { "code": "SYSC2004", ... }
  ],
  "count": 15, "query": "sysc"
}
```


Key: `sectionTypes` tells UI if course has labs, lectures, tutorials.

Data Acquisition: Where It All Comes Together

The course search bar shows data from all our sources

The User's View

Search Courses

🔍 sysc

SYSC1006 Foundations of Imperative Prog.

0.5 credits | LAB, LEC

SYSC2004 OO Software Development

0.5 credits | LAB, LEC

SYSC2100 Algorithms & Data Structures

0.5 credits | LAB, LEC, PAN

Where Each Field Comes From

UI Field	Data Source
SYSC1006	Timetable fetch
Course title	Courseblock parsing
0.5 credits	Courseblock parsing
LAB, LEC	Timetable section types

The payoff: All that data acquisition enables a simple, fast search showing everything students need.

Plus: Clicking a course shows prerequisite info.

Data Acquisition: The Real Challenge

Prerequisites are written in natural language — for humans, not computers

Here's What I Was Dealing With

Easy Example — COMP 1501:

```
Prerequisite(s): COMP 1005 or COMP 1405.
```

→ Clear: need one of two courses. I can parse this.

Medium Example — SYSC 1006:

```
Prerequisite(s): ECOR 1031 or (ECOR 1041 and ECOR 1042), all with minimum grade C-.
```

→ Tricky: OR and AND mixed, plus grade requirement. Needs careful conversion.

Hard Example — COMP 1405:

```
Prerequisite(s): restricted to students registered in the B.C.S. program, B.Cyber. program...
```

→ **No course requirements at all.** It's a program restriction.

Data Acquisition: The Conversion Problem I needed to turn prose into something a computer can evaluate

What the Parser Needs My prerequisite validator only understands symbolic logic:

```
SYSC3303              → single course
SYSC3303 && SYSC3310  → AND logic
SYSC3303 || SYSC3310  → OR logic
SYSC3303[C-]          → with grade requirement
FOURTH_YEAR           → year standing
```

Combine with parentheses: `(A || B) && C`

What I Had to Do **Manually** convert each prerequisite from prose to symbolic format.

Prose	Symbol
“and”	&&
“or”	`||`
“minimum grade of C-”	`[C-]`
“fourth-year standing”	`FOURTH_YEAR`
“permission of dept”	`PERMISSION`

348 courses. Each one read, understood, and converted by hand.

Data Acquisition: Conversion Examples From raw HTML to symbolic logic

Example 1: SYSC 1006 (Medium Difficulty)

Raw HTML:

```
Prerequisite(s): ECOR 1031 or (ECOR 1041 and ECOR 1042), all with minimum grade C-.
```

↓ Manual Conversion ↓

Symbolic Format:

```
"ECOR1031[C-] || (ECOR1041[C-] && ECOR1042[C-])"
```

Example 2: COMP 1405 (Program Restriction)

Raw HTML:

```
Prerequisite(s): restricted to students registered in the B.C.S. program...
```

↓ Manual Conversion ↓

Symbolic Format:

`"OTHER_REQUIREMENT"` — **We can't validate program restrictions programmatically (yet)**

Data Acquisition: What We Can't Parse

Some prerequisites don't fit into course logic

Types We Can Handle

- ✓ **Course requirements**
  "SYSC 3303 and SYSC 3310"
- ✓ **Grade requirements**
  "with a minimum grade of C-"
- ✓ **Year standing**
  "fourth-year standing"
- ✓ **Permission clauses**
  "or permission of the department"

Types We Can't (Yet)

- × **Program restrictions**
  "restricted to B.C.S. students"
- × **Credit thresholds**
  "15.0 credits completed"
- × **Vague equivalencies**
  "or equivalent experience"
- × **Instructor discretion**
  "at the discretion of..."

Solution: These become `OTHER_REQUIREMENT` or `PERMISSION` — the app flags them as "check manually".

Data Acquisition: From JSON to Database

The seed script parses symbolic expressions into tree nodes

Input: Symbolic Expression

```
"ECOR1031[C-] || (ECOR1041 && ECOR1042)"
```

Parser Logic (seed.ts)

```
function parseSimpleTerm(term: string) {
  if (['FIRST_YEAR','FOURTH_YEAR']
       .includes(term))
    return { type: 'YEAR_STANDING' }
  const gradeMatch = term.match(
    /^([A-Z]{3,4}\d{4})\[([A-Z\-+]*)\]$/)
  if (gradeMatch)
    return {type:'COURSE', grade: gradeMatch[2]}
  return { type: 'PERMISSION' }
}
```

Output: Tree Structure in DB

```
        [OR] ← root
       /    \
[COURSE]    [AND]
ECOR1031     /   \
          [COURSE] [COURSE]
```

Each box ⟶ row in `prerequisite_nodes`:

```
id | nodeType | requiredCourseId | parentId
1  | OR       | null             | null
2  | COURSE   | ECOR1031         | 1
```

Data Acquisition: Prerequisites Summary

The complete pipeline from course calendar to database



Why Manual Conversion?

- Natural language is **ambiguous**
- "all with a minimum grade" — which courses?
- **Accuracy matters more than automation**

The Result

- ✓ 348 courses with symbolic prereqs
- ✓ Grade & year requirements captured
- ✓ AND/OR logic preserved

The hard part is done. Now the validator can check any student's transcript.

Audit Parsing: Reliability & Trade-offs

Fast parsing with fallbacks for edge cases

Approaches Considered

Option	Trade-off
DOM parser (cheerio/ jsdom)	More robust, heavier dependency
Regex + fallbacks	Fast + simple, brittle if HTML changes

What We Implemented

- targeted patterns for stable fields
- fallback handling for missing grades/rows
- O(n) scan over the HTML

Example Patterns (Simplified)

```
/(\d{9})/                          → student ID
/Bachelor of Engineering - ([^<]+)/ → program
/<tr.*class="takenCourse".*>/g     → course rows
```

Known limitation: if Carleton changes page structure, we'll need to adjust patterns (or switch to a DOM parser).

Implementation 2/4: Prerequisite Representation

Store AND/OR logic once, evaluate many times

The Core Problem Prerequisites are logical expressions, not simple lists:

SYSC 4001 needs:
(SYSC 3303 AND SYSC 3310) OR permission
Fourth-year standing OR (AERO 2001 AND MATH 2004)

Implementation Goal Avoid re-parsing strings on every request. Store the structure in the DB so it's reusable and queryable.

Chosen Model: Tree Nodes

```
        [OR]
       /    \
   [AND]    [PERM]
   /   \
3303  3310

Node types: AND | OR | COURSE | YEAR | PERMISSION
```

Result: prerequisite checking becomes a fast tree walk

Prerequisite Evaluation: How It Runs

Recursive evaluation against the student's completions

Algorithm

```
eval(node):
  AND  → all children pass
  OR   → any child passes
  COURSE → course in transcript (and grade if required)
  YEAR → student standing >= required
  PERM → requires manual override
```

Complexity

- Time: O(n) for n nodes
- Space: O(h) recursion depth

Where It's Used

- course search / course detail: show "eligible vs missing prereqs"
- schedule building: filter out courses you can't take

Trade-off: a tree schema is more complex than a string, but it prevents repeated parsing and keeps the logic consistent everywhere.

Implementation 3/4: Schedule Generation

Generate conflict-free combinations automatically

Why This Needs an Algorithm A typical term quickly becomes combinatorial:

- 5 courses
- multiple sections per course
- LEC/LAB/TUT constraints

Goal (Implemented) Find many valid schedules (not just one) and stop early when a partial choice already conflicts.

Chosen Approach: Backtracking

```
pick course 1 section
  → try course 2 sections
    → if conflict: prune branch
    → else continue
```

Optimization: cap results (e.g., 50) to bound runtime

Schedule Conflicts: The Constraint

Detect time overlaps quickly while building schedules

Conflict Rule

Two meetings conflict iff:
same_day AND start1 < end2 AND start2 < end1

Why It Works

- O(1) overlap check
- called repeatedly during the backtracking search

What a "Section" Means

We treat a selected option as a bundle of required components:

- LEC must fit
- and LAB/TUT (if required) must fit

Trade-off: backtracking guarantees correctness, but worst-case runtime grows fast — pruning + caps keep it practical.

Implementation 4/4: Data Acquisition

Populate the DB from scattered Carleton sources

Reality Check There's no single public API:

- course calendar: many static HTML pages
- timetable: separate system
- electives: program-specific PDFs

Design Choice

Separate data ingestion from the web app (CLI/scripts populate DB; app only reads)

Pipeline (Implemented)

Data		How We Get It
Course titles + descriptions	←	parse calendar HTML pages
Prerequisite text	←	extract + convert to node tree
Section meeting times	←	import timetable export

Idempotent imports: rerun scripts without duplicating data

Data Evolution: What's Next

Keep data fresh each term and expand electives

In Place Today

- course + prerequisite ingestion into PostgreSQL
- term-aware section storage (Fall/Winter)
- clear split: scripts write, app reads

Constraints

- data changes every term
- must avoid hammering university systems

Remaining Work

- automate elective list ingestion
- improve multi-term (Fall/Winter/Summer) support
- admin tools for refresh + validation

This keeps the app accurate without manual re-entry each year.

Design Principle: User-Centered Design

Making It Simple — Before vs After

The Old Way

"Can I take SYSC 4001?"
→ Open audit in one tab
→ Open calendar in another
→ Find the prerequisites
→ Check each one against your audit
→ Figure out the AND/OR logic

Repeat for every course

With Our App

"Can I take SYSC 4001?"
→ Type "SYSC 4001"
→ See "You're good" or "Need SYSC 3303"

Instant feedback

Upload once, and the app remembers what you've completed

Design Principle: Constraint Satisfaction

Too many combinations to check by hand

The Problem

- Pick 5 courses
- Each course may have multiple sections
- Each section may have LEC, LAB, TUT components
- All combinations need to be checked for time conflicts

Doing this manually is tedious and error-prone.

How We Do It: Backtracking

```
generateSchedules(courses, current):
  if all courses scheduled:
    save it, return
  for section in nextCourse:
    if no conflict:
      add section, recurse, remove
    else: skip this branch
```

If something conflicts, stop early and try the next option. This saves time early.

Design Principle: Separation of Concerns

Three layers, each does one job

Frontend — React + TypeScript

What you see and interact with

↓ talks to ↓

Backend — Express + Node.js

Handles requests, runs the logic, talks to the database

↓ queries ↓

Database — PostgreSQL

Stores courses, sections, and prerequisite rules

Design Principle: Data Modeling

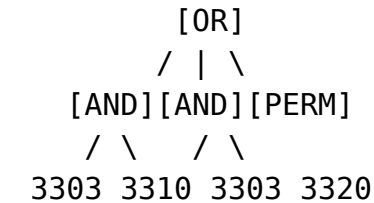
Prerequisites aren't just a list — they're logic

The Problem

Prerequisites have AND/OR conditions:

> SYSC 4001 needs:
> (SYSC 3303 AND SYSC 3310)
> OR (SYSC 3303 AND SYSC 3320)
> OR (Permission of instructor)

Our Solution: A Tree



Walk the tree, check each node. If any OR branch passes, you're good.

Design Principle: Performance Optimization

Don’t hammer the database

Without Optimization

User types “SYSC4001”
⟶ 8 keystrokes = 8 API calls
⟶ 8 database queries
Multiply by many concurrent users…

What We Do Instead

Debounce — wait 300ms after typing stops
Cache — remember results for 5 min
Min length — don’t search for “a”
Cancel — abort stale requests

Scenario	Before	After
Typing “SYSC4907”	8 requests	1 request
Same search twice	2 requests	0 (cached)

Work in Progress: Electives

The Annoying Part

Right now if you want to know what Science Electives you can take as a Software Eng student, you have to go to a separate website and dig through PDFs.

Same deal for Complementary Studies, tech electives, etc.

Where We're At

- We detect elective types from your audit
- Currently using placeholder data

What We're Adding

1. Add elective tables to the DB
Which electives count for which programs

2. Get the real data
From Carleton's official lists

3. Suggest electives automatically
Based on your program and year

4. Include them in schedule generation
Pick a Science Elective → see all options

Tech Stack

Layer	Tech	Why
Frontend	React + TypeScript	Components, type safety
Styling	TailwindCSS	Fast to write, looks consistent
Backend	Express + Node.js	Simple, fast, JS everywhere
Database	PostgreSQL	Solid, handles complex queries
ORM	Prisma	Type-safe DB queries, easy migrations

How It Works

From upload to registration

1. Upload
Drop in your audit HTML

→

2. Parse
We extract your courses

→

3. Search
Find courses, check prereqs

4. Generate
Click the button

→

5. Pick One
Browse the timetables

→

6. Register
Copy CRNs, paste in Central

Problems We Ran Into

Problem	How We Solved It
Prerequisites have complex AND/OR logic	Store them as trees, walk recursively
Many schedule combinations to check	Backtracking with early exit on conflicts
Search hitting DB too much	Debounce, cache, cancel old requests
Audits aren't all formatted the same	Pattern matching with fallbacks
Page refresh loses your state	URL routing + sessionStorage
Checking time conflicts	Interval overlap check, filter by day first

What's Left (2 Months)

Electives (Main Focus)

- Get official elective lists into the DB
- Link them to programs
- Auto-suggest based on your audit
- Include in schedule generation

UI Polish

- Better way to compare schedules
- Improve visual feedback

Term Selection

- Add ability to pick Fall vs Winter
- Fetch section data for both terms

Testing & Edge Cases

- End-to-end tests
- Handle weird audit formats
- Make sure it doesn't break

Summary

What It Does

- ✓ Upload your audit once, we remember it
- ✓ Search any course, see if you qualify
- ✓ Generate conflict-free schedules
- ✓ Copy CRNs in one click

How We Built It

- Focused on making it simple to use
- Backtracking to handle the combinatorics
- Clean separation: frontend / backend / DB
- Trees for prerequisite logic
- Debouncing + caching so search is fast

Thanks!

Questions?

Project: SYSC 4907 Capstone
Supervisor: Professor Jerome Talim
Stack: React, Express, PostgreSQL, Prisma, TypeScript